

Diseño y Análisis de Algoritmos

Prueba Sumativa 2

Profesor: Felipe Ruiz

27 de Noviembre, 2025

Nombre: _____ Rut: _____

Indicaciones

- Puntaje total: **80 puntos**. Duración: **80 min**. Entrega de resultados: **4 de Diciembre, 2025**.
- Esta evaluación tiene 4 páginas (incluyendo la portada) y 3 preguntas. Compruebe que dispone de todas las páginas. Responda a las preguntas en los espacios previstos para ello en las hojas de preguntas. Asegúrese de marcar apropiadamente sus respuestas.
- Durante la prueba no se puede utilizar: teléfono móvil, apuntes y/o computador. Está prohibido intentar conectarse a internet de cualquier manera. Si es sorprendido obtendrá la calificación mínima. Tampoco puede utilizar dispositivos de almacenamiento externos o cualquier otro dispositivo, relojes inteligentes, etc.
- Lea la prueba completamente **DOS** veces antes de hacer cualquier pregunta.
- Una prueba respondida correctamente en un 60%, de acuerdo con las ponderaciones asignadas, corresponde a una nota 4,0.
- Solamente se pueden realizar preguntas durante los primeros 10 minutos de la prueba. Solo se responderán preguntas respecto a los enunciados a viva voz.
- La prueba es individual; cualquier sospecha de copia será calificada con la nota mínima y el caso será remitido al comité de ética.
- En su espacio personal no debe haber nada más que hojas de papel en blanco, lápiz, goma y/o calculadora.
- El resto de sus implementos debe guardarlos dentro de su mochila/bolso y ésta debe posicionarse al frente debajo de la pizarra. *Si leyó hasta este punto, felicidades: para saber que lo hizo, dibuje una estrella al final de esta página.*

Acepto las condiciones firmando: _____

1. Hashing con muchas colisiones**(15 puntos)**

Considere una tabla hash donde las colisiones se resuelven por *encadenamiento*, pero debido a una mala elección de la función de hash o a un factor de carga muy alto, algunas casillas llegan a tener listas de colisión **muy largas** (longitud k grande).

Para cada casilla de la tabla se evalúan dos implementaciones posibles para almacenar las claves que caen en ella:

- **Versión A:** Lista simplemente enlazada *no ordenada*; las búsquedas se realizan de forma lineal recorriendo la lista.
- **Versión B:** Arreglo dinámico que se mantiene siempre *ordenado* de menor a mayor clave, donde las búsquedas se realizan usando *búsqueda binaria* sobre el arreglo.

Sea k la cantidad de claves almacenadas en una casilla dada (es decir, el largo de la lista/arreglo en esa posición). Suponga que, por las malas colisiones, k puede ser grande.

- (a) Compare asintóticamente, en función de k , el costo de las siguientes operaciones en la Versión A y la Versión B: **búsqueda**, **inserción** y **eliminación** de una clave. Use notación $O(\cdot)$ y justifique brevemente cada caso, comentando el rol de la búsqueda lineal vs. binaria y de los desplazamientos en el arreglo. **(10 puntos)**
- (b) En base a su análisis, discuta en 2–3 líneas en qué situaciones podría ser conveniente mantener el arreglo ordenado (Versión B) a pesar de que algunas operaciones sean más costosas. Relacione su respuesta con el hecho de que k puede ser grande debido a muchas colisiones. **(5 puntos)**

2. Búsqueda en profundidad**(20 puntos)**

Considere el grafo dirigido mostrado en la Figura 1. Utilizando el algoritmo de búsqueda en profundidad (*Depth-First Search*, DFS), determine para cada vértice v :

- el tiempo de descubrimiento $d[v]$: momento en que el vértice es visitado por primera vez;
- el tiempo de finalización $f[v]$: momento en que se termina de explorar completamente el vértice.

Considere que:

- el procedimiento DFS recorre los vértices en orden alfabético;
- para cada vértice, su lista de adyacencia también está ordenada alfabéticamente.

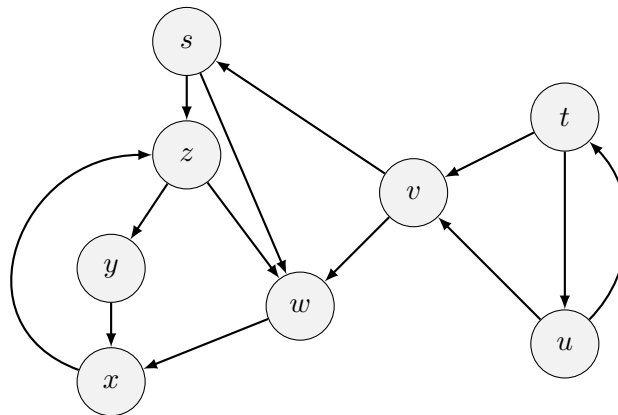


Figura 1: Grafo dirigido

3. Distancia mínima en la red de Metro de Santiago

(45 puntos)

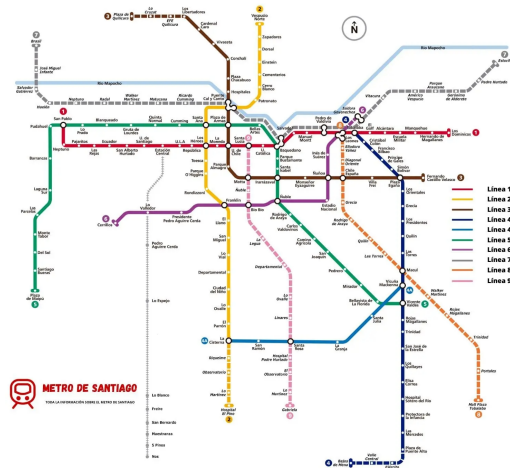


Figura 2: Red de Metro de Santiago.

Su compañero de trabajo, Juan, vive cerca de la estación **Santa Ana** y trabaja cerca de la estación **Ñuñoa**. Entre estas estaciones existen múltiples rutas posibles en Metro; sin embargo, a Juan solo le interesa la ruta en la que deba pasar por la **menor cantidad de estaciones** posibles. A las 7:00 de la mañana, medio dormido en el andén, Juan se acuerda de que tú estudiaste computación y llega a la oficina a desafiarte a implementar un algoritmo para resolver este problema.

Para modelar este problema, considere la red de Metro de Santiago como un grafo simple, no dirigido y no ponderado $G = (V, E)$, donde cada vértice representa una estación y cada arista $(u, v) \in E$ indica que existe un tramo directo de vía entre las estaciones u y v .

Dadas dos estaciones $s, t \in V$ (por ejemplo, $s = \text{Santa Ana}$ y $t = \text{Ñuñoa}$), se desea calcular la distancia mínima en número de “saltos” entre s y t , es decir, la longitud del camino más corto (en número de aristas) que conecta s con t .

- Diseñe un algoritmo que, dados G , s y t , compute la **cantidad mínima de estaciones** que Juan debe recorrer para ir desde s hasta t , o determine que no existe un camino entre ellas. Su algoritmo debe ser correcto para cualquier grafo simple, no dirigido y no ponderado, y debe ejecutarse en tiempo *lineal* en el tamaño del grafo (es decir, $O(|V| + |E|)$). **(30 puntos)**
- Suponga ahora que ya ejecutó un algoritmo sobre G que, para cada vértice $x \in V$, almacenó en x la **distancia mínima** desde s . Usando el grafo G obtenido del problema anterior, específicamente el predecesor $\pi[v]$, diseñe el procedimiento

PRINT-PATH(G, s, v)

que **imprime en orden** una secuencia de estaciones que corresponde a *uno de los caminos más cortos* desde s hasta un vértice v . Describa claramente la idea de su solución, escriba el pseudocódigo de PRINT-PATH y justifique brevemente que su complejidad temporal no excede $O(|V| + |E|)$. **(15 puntos)**